Geometric and Learned Relay Control for Connectivity-Constrained Drone Delivery

Gwang-Jong Ko[a], Taesu Cheong[a,*]

[a]*School of Industrial and Management Engineering, Korea University, 145, Anam-ro, Seongbuk-gu, Seoul, 02841, Republic of Korea*

Abstract

In disaster relief, delivery drones must stay connected to a control centre through a multi-hop relay chain formed by the fleet itself, while carrying supplies with limited capacity and returning to reload. We model this as a makespan-minimisation problem in which every drone can either deliver or relay, and connectivity is a hard constraint enforced at every time step. We propose two controllers that share the same connectivity-preserving projection: a geometric chain-relay heuristic with a deadlock-escape mechanism, and a fleet-size-agnostic policy learned once with discrete soft actor-critic over a set-based, autoregressive manager. Both are compared on identical instances with a paired protocol across fleets of 3 to 12 drones and 20 to 400 destinations, against four learning-free baselines and a time-expanded mixed-integer reference model on small instances. The learned policy completes every instance in every configuration; the heuristic completes 98–100% with the escape mechanism and 62% without it. The two controllers divide the configuration space: with three drones the heuristic is 30–39 steps faster in paired median, with four drones the two tie over 480 rollouts, and with five or more drones the learned policy is faster by 38 to 178 steps; the policy is trained only on three and four drones and generalises to every larger fleet without retraining. The three-drone deficit is localised to a single decision, which drone relays, and ten policy-side interventions covering information, credit assignment, decision structure, initialisation and reward shaping all fail to remove it; half of it disappears when the connectivity projection is changed from radial shrinking to tangential sliding, showing that the deficit is partly a property of how the constraint is enforced rather than of the learner. Both controllers run in milliseconds per decision, three orders of magnitude below exact optimisation, which does not return a plan for ten destinations within ninety minutes.

Keywords: drone delivery, relay network, connectivity constraint, multi-agent reinforcement learning, heuristic, disaster logistics

1. Introduction

When communication, logistics and road infrastructure are destroyed, a fleet of drones can carry emergency supplies from a temporary control centre to many destinations. Two features make this different from parcel delivery. First, every drone must be reachable by the control centre at all times, because loading and unloading are supervised remotely and the delivery status must be tracked; with the ground network gone, reachability is provided only by the drones themselves acting as radio relays in a multi-hop chain. Second, the fleet is small and each drone carries a few units, so drones shuttle between the centre and the destinations and every drone that stands still as a relay is a drone that is not delivering. The operational question is how to assign delivery and relay roles over time so that the last destination is served as early as possible without ever breaking the chain.

Three properties make the problem hard. A hard connectivity constraint creates deadlocks: a greedy assignment that sends every drone to its nearest destination completes none of the standard instances in our study, because the fleet pulls itself apart until no drone can move. Relaying competes directly with throughput, so the right number and identity of relays changes with the geometry of the remaining destinations and with how many drones are left with cargo. And exact optimisation does not scale: a time-expanded mixed-integer model on a 98-metre lattice returns plans for four and six destinations but no feasible plan for ten within ninety minutes.

Proposal. We propose two controllers for this problem and study them side by side.

1. A *geometric chain-relay heuristic*. The drone farthest from the centre with cargo is the lead; the number of relays is the number of communication hops its target needs; the relays are the next-farthest drones, stationed on the segment from the centre to the lead at intervals of $0.9R$. Because the rule is deterministic it can repeat a deadlocked assignment indefinitely, so we add a *deadlock-escape* mechanism that swaps the lead whenever the fleet has stalled for a fixed number of steps. The escape mechanism alone lifts completion on the standard configuration from 62% to 98%.
2. A *learned policy* that is agnostic to fleet size. A manager assigns each drone one of seven discrete options every twenty steps; it encodes the drones, candidate destinations and relay slot as a set, pools them with attention, and decides autoregressively so that a formation of one lead and k relays can be composed deliberately. It is trained once with discrete soft actor-critic on fleets of three and four

*Corresponding author.

Email addresses: koptimizer@korea.ac.kr (Gwang-Jong Ko), tcheong@korea.ac.kr (Taesu Cheong)

drones and then run without retraining on fleets of up to twelve.

Both controllers act through the same connectivity-preserving projection: each commanded move is scaled by the largest factor that keeps the fleet connected. Connectivity is therefore guaranteed by construction for both, and the empirical question is whether a controller makes progress under the constraint rather than whether it violates it.

Research questions. We organise the study around four questions.

RQ1 Do the proposed controllers always complete the delivery while connectivity is enforced at every step, and when they do not, why not?

RQ2 On small instances, how close are the proposed controllers to a reference plan obtained by exact optimisation of a lattice model?

RQ3 How does the performance of the two controllers differ across environments, in particular across fleet size, number of destinations, depot position and communication radius; does a policy trained on small fleets generalise to larger ones; and where do the differences come from?

RQ4 Are the components of each controller necessary?

Findings. The learned policy completes every instance in every configuration tested; the heuristic completes 98–100% with the escape mechanism (RQ1). On instances small enough for the lattice model, both controllers are within a few percent of the reference plan, and the reference plan itself is a restriction rather than a bound of the continuous problem (RQ2). The two controllers divide the configuration space by fleet size: the heuristic is faster with three drones, the two tie with four, and the learned policy is faster with five or more, by 62 to 178 steps up to ten drones and 76 at twelve, although it never saw more than four drones in training; the three-drone deficit is a single decision, which drone relays, and it resists ten policy-side interventions but shrinks by half when the connectivity projection is changed (RQ3). Removing rule regularisation, configuration randomisation or entropy annealing from the learned policy costs hundreds of steps or completion, and removing the escape mechanism from the heuristic costs a third of its completions (RQ4).

Contributions. (i) A geometric chain-relay heuristic with deadlock escape and a fleet-size-agnostic learned policy for connectivity-constrained delivery, both built on a projection that makes the constraint hold by construction. (ii) A paired evaluation across fleet sizes that shows the two controllers are complementary, together with the mechanism behind the boundary. (iii) A set of ten documented negative results on closing the scarce-fleet deficit by changing the learner, and a sensitivity result showing that half of the deficit is attributable to the enforcement model. (iv) A lattice MILP reference with an honest account of what it does and does not bound.

2. Related Work

Drone delivery routing. Routing with drones has been studied since the flying-sidekick travelling salesman problem [1] and its multi-drone extensions [2, 3, 4]. These models treat communication as free: a drone is never required to be reachable, and no vehicle gives up its delivery role to serve as a relay. Learned solvers for min-max multi-vehicle routing [5, 6] show that set-based policies generalise across problem sizes, which is the property we need across fleet sizes, but again without a connectivity constraint. Dual-mission fleets in which the same drones deliver and sense [7] formalise a per-delivery detour budget; the secondary task there is sensing, not communication.

UAV relays and connectivity maintenance. Aerial relay positioning is its own literature [8, 9, 10], and joint optimisation of mission and relay UAVs with multi-agent learning exists [11, 12]. In these works the relays are the mission, or a dedicated sub-fleet with its own reward; they do not carry cargo, do not reload, and do not minimise a delivery makespan. Connectivity maintenance for multi-robot navigation has been approached with learned controllers [13]. Switching between a task role and a relay role is an established idea in multi-robot exploration [14, 15], where relay robots land and act as fixed nodes.

Where this work sits. To the best of our knowledge, no prior work combines (i) a homogeneous fleet carrying physical cargo to many destinations with capacity and reloading, (ii) a hard requirement that every drone be connected to the centre by a multi-hop chain at every step, and (iii) a makespan objective. In this setting relay duty competes with cargo throughput and the fleet must return to the depot repeatedly, which is what makes role assignment a sequential decision rather than a one-off placement.

Learning machinery. On the learning side we build on soft actor-critic [16], centralised critics for decentralised execution [17, 18], value decomposition [19, 20], options as a semi-Markov decision process [21], invalid-action masking [22] and safe-action projection [23]. Our negative results in Section 6.3 test several of these ideas directly: per-agent credit, joint-action critics, monotonic value mixing and potential-style shaping [24] are all tried as remedies for the scarce-fleet deficit and none of them removes it.

3. Problem Statement

A control centre sits at $c \in [0, 1000]^2$ metres and M destinations $d_1, \ldots, d_M$ lie within $0.9\,NR$ of it, where N is the number of drones and $R = 300$ m the communication range. Each drone flies at $v = 13.89$ m/s, carries $Q = 5$ units, unloads one unit at a destination in $S = 10$ steps and reloads to Q at the centre in S steps; a destination or the centre is reached when a drone is within 25 m of it, and one step is one second. Every destination needs exactly one unit. Drones do not collide, are not affected by wind, and battery is not modelled.

Connectivity. Let G_t be the graph whose vertices are the centre and the N drones and whose edges join vertices within R of each other. The fleet is *connected* at step t if G_t is connected, which is the same as saying that every drone has a multi-hop path to the centre. The constraint is that G_t is connected at every t. It is enforced rather than penalised (Section 4.1).

Objective. The makespan is the step at which the last destination is served. A controller *completes* an instance if it serves all destinations within a budget of 10,000 steps. We report completion rate and, over completed instances, makespan.

Configurations. A configuration is a triple of fleet size N, number of destinations M and depot placement (fixed at the map centre, or uniformly random). The *standard* configuration is $N = 4$, $M = 50$ with the fixed depot. The learned policy is trained with $N \in \{3, 4\}$, $M \in [20, 80]$ and a random depot; all other configurations are unseen by it. The heuristic has no training.

3.1. Exact reference: a time-expanded lattice model

For small instances we also formulate the problem as a mixed-integer program so that the two controllers can be compared with a plan obtained by exact optimisation. The continuous problem is not directly tractable, so we discretise time and space and keep the connectivity requirement exact on the discretisation.

Discretisation. A period is $\kappa = S = 10$ steps, so a service occupies exactly one period and a manager decision of the learned policy spans two; the per-period flight range is $\ell = v\kappa$. Space is a square lattice G of spacing $h = \ell/\sqrt{2} = 98.2$ m over the bounding box of the instance, from which nodes farther than $0.9NR$ from the centre are deleted; the centre and every destination are added as exact nodes, so the node set is $N_v = \{0\} \cup D \cup G$ with the centre indexed 0 and D the destinations. With this spacing all eight king moves are feasible in one period. The move set and the communication set are

$$A(i) = \{j : \|p_i - p_j\| \le \ell\} \ni i, \qquad E = \{(i, j) : i \ne j,\ \|p_i - p_j\| \le R\}. \tag{1}$$

Axis-aligned moves cover h rather than ℓ per period, so the lattice slows a drone by up to 29% in those directions; this discretisation error always acts against the model.

Variables. For drones $k \in K$ and periods $t \in T = \{0, \dots, \bar{T}\}$: $y_{ikt} \in \{0, 1\}$ is one if drone k is at node i at the boundary of period t; $s_{jkt} \in \{0, 1\}$ if drone k delivers to destination j during period t; $r_{kt} \in \{0, 1\}$ if it reloads at the centre during period t; $q_{kt} \in \{0, \dots, Q\}$ is its cargo at the boundary of period t and $\rho_{kt} \in [0, Q]$ the cargo taken on during the period; $g_{ijt} \ge 0$ is a connectivity flow on arc $(i, j) \in E$ in period t; and C is the makespan in periods. The horizon $\bar{T}$ is set from the heuristic's makespan with 15% slack.

Model.

$$\min \ C + \varepsilon \textstyle\sum_{i \ne 0} \sum_k \sum_t y_{ikt} \tag{2}$$

$$\text{s.t.} \ \ \textstyle\sum_i y_{ikt} = 1, \ \ y_{0k0} = 1 \qquad \forall k,t \tag{3}$$

$$y_{i,k,t+1} \le \textstyle\sum_{j \in A(i)} y_{jkt} \qquad \forall i,k,t<\bar{T} \tag{4}$$

$$\textstyle\sum_k \sum_{t<\bar{T}} s_{jkt} = 1 \qquad \forall j \in D \tag{5}$$

$$s_{jkt} \le y_{jkt}, \ \ s_{jkt} \le y_{j,k,t+1} \qquad \forall j \in D,k,t \tag{6}$$

$$r_{kt} \le y_{0kt}, \ \ r_{kt} \le y_{0,k,t+1} \qquad \forall k,t \tag{7}$$

$$\textstyle\sum_{j \in D} s_{jkt} + r_{kt} \le 1, \ \ \sum_{j \in D} s_{jkt} \le q_{kt} \qquad \forall k,t \tag{8}$$

$$q_{k,t+1} = q_{kt} - \textstyle\sum_{j \in D} s_{jkt} + \rho_{kt}, \ \ q_{k0} = Q \qquad \forall k,t \tag{9}$$

$$\rho_{kt} \le Q\, r_{kt}, \ \ q_{k,t+1} \ge Q\, r_{kt} \qquad \forall k,t \tag{10}$$

$$\textstyle\sum_{j:(i,j) \in E} g_{ijt} - \sum_{j:(j,i) \in E} g_{jit} = u_{it} \qquad \forall i \ne 0,t \tag{11}$$

$$g_{ijt} \le |K|\, u_{it} \qquad \forall (i,j) \in E,t \tag{12}$$

$$C \ge t\, s_{jkt} \qquad \forall j \in D,k,t<\bar{T} \tag{13}$$

$$s_{jkt} = 0 \qquad \forall k, \ j \in D: \ \mathrm{idx}(j) < k, \ t<\bar{T} \tag{14}$$

where $u_{it} = \sum_k y_{ikt}$ is the occupancy of node i with $u_{0t} := 1$, and $\varepsilon = 10^{-5}$ is a tie-break that prefers keeping idle drones at the centre and cannot change the optimal makespan. Constraints (3)–(4) place every drone at exactly one node per period boundary, starting at the centre, and allow a move only to a node within one period's range. Constraints (5)–(10) serve each destination once, require the serving drone to be at the destination for the whole period (the service freeze), make reloading fill the drone completely, and track cargo. Constraints (11)–(12) encode connectivity as a single-commodity flow in every period: each occupied node ships one unit towards the centre, the only sink, and an arc may carry flow only if it leaves an occupied node. This system is feasible if and only if every occupied node is joined to the centre by a path of occupied nodes whose consecutive distances are at most R, which is exactly the simulator's multi-hop test; the symmetric head-capacity rows are redundant and omitted, halving the dominant row count. Constraint (13) defines the makespan and (14) is a valid symmetry-breaking cut for identical drones.

What the optimum means. Because drones are confined to lattice nodes and axis moves are shortened, the model's optimum is an *upper bound* on the continuous optimum, and its dual bound is a bound on the lattice problem only. A certified lower bound through a cell relaxation, in which a lattice node stands for its cell and communication is tested by the minimum distance between cells, was attempted and abandoned: it inflates a three-hop chain's reach from 900 m to 1,317 m and returns bounds below the trivial analytic bound. We therefore call the solver's value a *reference plan*, replay every plan in the continuous simulator, and report the replayed makespan.

4. Proposed Controllers

Figure 1 summarises the setting and the two controllers. Both controllers decide at the level of *goals*: at every decision point each drone is given a goal (a destination, the centre, or a relay slot) and flies towards it in a straight line at full speed. Decisions are revised every 20 steps, or immediately when a goal

becomes invalid (the destination has been served by another drone, the drone has run out of cargo, or the relay slot has disappeared because the chain shortened). The option set is the same for both: the five candidate destinations (the three nearest unserved ones and the two farthest from the centre), return to the centre, and relay. Relay slots lie on the segment from the centre to the lead drone at multiples of $0.9R$ and are assigned by the environment, so a controller decides only *whether* a drone relays.

4.1. Connectivity-preserving projection

A reward penalty for disconnection would punish situations that no action can prevent, for example a drone that is stationary while unloading and is stranded when a teammate departs. We therefore enforce the constraint by projection. Given the commanded displacements of all drones, the projection finds, drone by drone, the largest scaling factor in $[0, 1]$ under which the fleet remains connected, choosing at each round the drone whose restriction leaves the most drones connected and breaking ties at random so that no anchor drone is starved. A drone whose displacement is scaled below 10% is counted as *stalled* for that step. The previous positions are always connected, so a zero scaling is a valid fallback and the constraint holds by construction. We refer to this as *radial* projection because the displacement keeps its direction and loses magnitude; Section 6.3 studies an alternative that keeps the tangential component.

4.2. Geometric chain-relay heuristic with deadlock escape

Assignment rule. Let r_i be the distance of drone i from the centre. The *lead* is the drone with cargo that is farthest from the centre. Its target is its nearest unserved destination, at distance d from the centre; the chain needs $h = \lceil d/(0.9R) \rceil$ hops and therefore $h-1$ relays, capped at $N-1$. The $h-1$ non-lead drones farthest from the centre are assigned to relay; every other drone with cargo goes to its nearest unserved destination and every drone without cargo returns to the centre. The rule is evaluated at every decision point and costs $O(N \log N)$.

Deadlock escape. The rule is deterministic, so when a geometry stalls the fleet it re-issues the same assignment and the stall persists: without an escape mechanism the rule completes 62% of standard instances. We detect a *partial deadlock* when, among the drones that attempted to move, all but at most one were stalled by the projection. When a partial deadlock has persisted for k_{stall} steps, the controller enters an escape phase of h_{hold} steps during which the lead is replaced by the next-farthest drone with cargo and the former lead returns to the centre; on the n-th escape of an episode the n-th farthest drone becomes the lead, so that repeated escapes cycle through the fleet. An alternative escape mode, in which every drone except the lead retreats towards the centre to slacken the chain, is also implemented. The values $k_{\text{stall}} = 120$, $h_{\text{hold}} = 20$ and the lead-swap mode are selected in Section 6.4 on seeds disjoint from the evaluation seeds; this setting is used in every comparison in this paper.

4.3. Fleet-size-agnostic learned policy

Decision structure. The learned controller is a manager that emits one of the $K = 7$ options for each drone at each decision point; the low-level controller is the same straight-line flight as the heuristic. A learned low-level controller was tried in earlier versions and reduced completion to 0–2%, so it is not used. Between decision points the environment is a semi-Markov decision process [21] and the manager's transitions store the discounted return of the option.

Set-based encoder. Each drone is encoded from four kinds of entities: its own state (10 features: cargo, distance to the centre, hop depth, remaining-destination fraction, time, movement ratio, stall indicators, service timer, cut-vertex indicator), its five candidate destinations (6 features each: distance, direction, whether a teammate already targets it, distance from the centre, hops required), its peers ($N - 1$ entities of 9 features: relative position, cargo, hop depth and a one-hot of the peer's current role) and its relay slot (4 features). Distances are normalised by R, not by the map size. Each entity type passes through a two-layer MLP of width 128; the drone's own embedding is used as the query of two multi-head attention layers that pool the peer set and the candidate set; the four vectors are concatenated and fused. A pointer head scores the five candidates against the fused embedding and a fixed head scores *return* and *relay*. Nothing in the network depends on N or M, which is what allows one set of weights to run on fleets it never saw. Invalid options are masked [22].

Autoregressive decisions. Drones are ordered by distance from the centre, farthest first; after each drone's option is sampled its new role is written into the peer features seen by the drones decided after it. A formation of one lead and k relays can arise only by coincidence under independent per-drone sampling, but can be composed deliberately under sequential decisions. The price is N sequential network calls per decision.

Training. The manager is trained with discrete soft actor-critic [16]. A twin-Q critic with attention across drone embeddings outputs per-drone action values whose mean over drones is regressed on the team return; the actor loss is the standard expectation form. The team reward is +10 per delivery, +30 per reload, $-0.01N$ per step and a residual penalty of $-0.1N$ times the fraction of destinations still unserved; the discount is $\gamma = 0.997$ per step. Three ingredients matter. (i) *Entropy target annealing*: the conventional target of $0.6 \log n_{\text{valid}}$ keeps the policy too random to hold a multi-step formation, while that randomness is also the only mechanism by which a deadlocked fleet escapes; annealing the target linearly from $0.6 \log n$ to $0.25 \log n$ over the first 400 episodes lowered the standard-configuration makespan from 1,418 to the 900s. (ii) *Rule regularisation*: a cross-entropy term with weight 2.0 pulls the policy towards the action the heuristic's assignment rule would take, wherever that action is unmasked. (iii) *Configuration randomisation*: N, M and the depot are resampled every episode. Optimisation uses Adam at 3×10^{-4}, batches of 256 option transitions from a replay buffer of 60,000, and Polyak

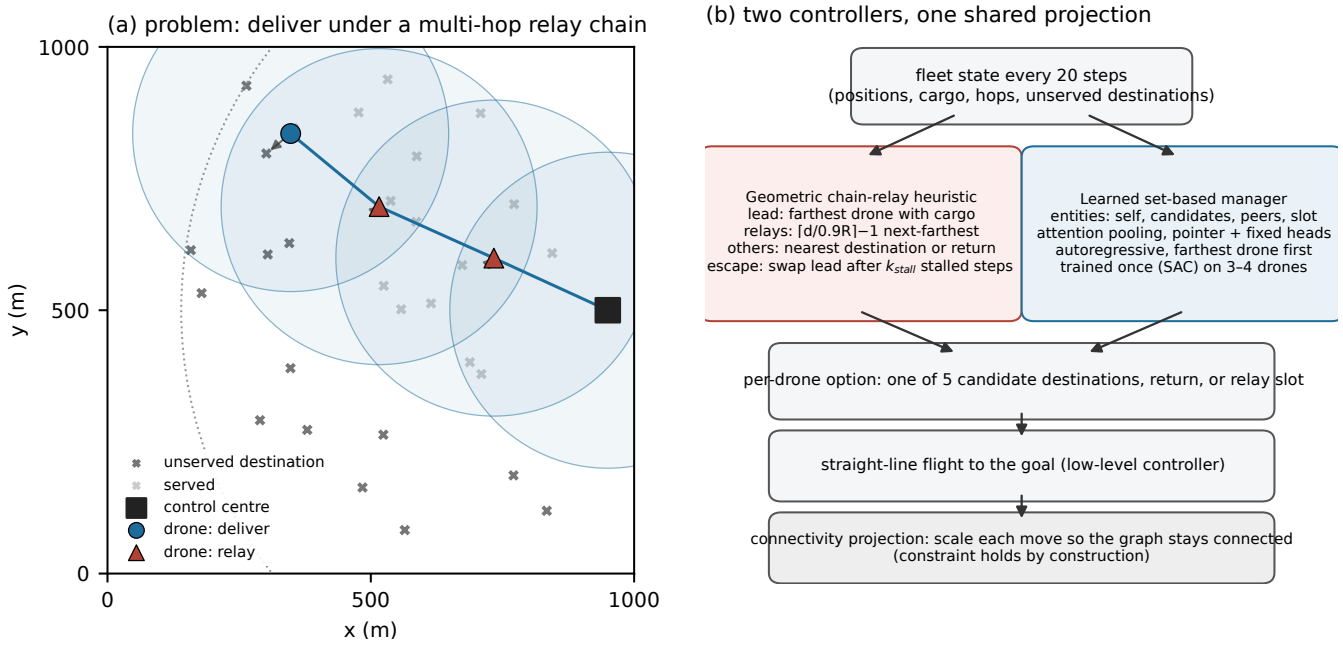


Figure 1: (a) An instance with three drones after 220 steps under the heuristic: the lead drone (blue circle) heads for a far destination, one drone relays (red triangle) on the segment to the centre, and one returns to reload; shaded discs are communication ranges and the dotted circle is the reachable region $0.9NR$. (b) The two controllers emit the same per-drone options and share the straight-line low-level controller and the connectivity projection.

averaging with $\tau = 0.005$. Checkpoints are scored every 50 episodes on 80 held-out instances with a 3,000-step budget by $100f + 40(1 - \bar{t}/3000)$, where f is the completion rate and $\bar{t}$ the mean steps, so that one percentage point of completion is worth 75 steps; training stops after eight evaluations without improvement or at 1,000 episodes. One training run takes about 13 hours on a single RTX 5080 when four runs share the GPU.

5. Experimental Setup

Instances and configurations. Instances are generated from integer seeds; a seed, N, M and the depot mode determine the instance exactly, so two controllers can be run on identical instances. Evaluation seeds are disjoint from the seeds used for training, for checkpoint selection and for tuning the heuristic. We evaluate on the standard configuration ($N = 4$, $M = 50$, fixed depot) and on eight unseen configurations with random depots: $N/M \in \{3/20, 3/30, 4/80, 5/50, 6/80, 8/150, 10/300, 12/400\}$.

Baselines. Besides the two proposed controllers we run a greedy assignment with no relays, two fixed-relay policies that permanently dedicate one or two drones, and the geometric rule without the escape mechanism. These establish that the problem is not solved by naive assignment.

Reference model. Small instances are solved with the lattice model of Section 3.1 using Gurobi 13 [25] with six threads; the time limits are given with the results in Section 6.2. Every returned plan is replayed in the continuous simulator.

Paired protocol. Averaging only the episodes each method completes favours the method that gives up on hard instances, and a single long stall moves a 40-rollout mean by over 100 steps. All comparisons between the two controllers are therefore *paired*: both run on the same instance, the difference is taken over instances both complete, and we report the median paired difference, the win–loss count and a Wilcoxon signed-rank p-value. Inference samples from the policy; deterministic argmax inference has the same median but a far heavier tail. The standard configuration uses 480 rollouts (60 seeds, 8 repetitions, two independent measurement runs), the 3/30 and 5/50 configurations 200 (40 seeds, 5 repetitions) and the remaining configurations 80 (40 seeds, 2 repetitions). Repetitions differ because the projection's tie breaking is random. We found 40 rollouts too few to support any claim about tails.

6. Results

6.1. RQ1: Completion under the hard constraint

Connectivity was never violated in any training episode or evaluation rollout, as the projection guarantees. Table 1 reports completion within the 10,000-step budget. The learned policy completes every instance in every configuration. The heuristic with escape completes 98–100%; its misses concentrate where drones are plentiful, where a long chain that is not needed still gets stationed and stalls. Without the escape mechanism the rule completes 61.7% of standard instances, and the naive baselines complete almost none: the greedy assignment pulls the fleet apart, and fixed relay counts are wrong for most instances because the number of relays a destination needs varies with its distance.

Table 1: Completion within the 10,000-step budget. Learning-free baselines on 120 (standard) or 40 (unseen) rollouts; the two proposed controllers on the paired rollout counts of Section 5.

Method	4/50 standard	3/30	5/50
Greedy, no relay	0%	0%	25%
One fixed relay	0%	2.5%	40%
Two fixed relays	3.3%	52.5%	65%
Rule, no escape	61.7%	95%	92.5%
Rule + escape (proposed)	473/480	200/200	191/200
Learned (proposed)	**480/480**	**200/200**	**200/200**
	3/20	4/80	6/80
Rule + escape (proposed)	80/80	80/80	79/80
Learned (proposed)	**80/80**	**80/80**	**80/80**
	8/150	10/300	
Rule + escape (proposed)	80/80	80/80	
Learned (proposed)	**80/80**	**80/80**	

6.2. RQ2: Distance to the lattice reference plan

Table 2 and Figure 2 compare the two controllers with the lattice reference on 34 instances with three drones: ten each for $M \in \{4, 6, 8\}$ and four for $M = 10$, with time limits of 30, 60, 120 and 120 minutes and six solver threads. Every plan the solver returns is replayed in the continuous simulator, and the replayed makespan is the number we compare against; one plan for $M = 8$ delivered only seven of eight destinations on replay and is counted as found but not replayed.

Three observations follow. First, the solver's reach ends between six and eight destinations: it returns plans for 9 of 10 instances with four destinations (four of them certified optimal on the lattice), 8 of 10 with six, 3 of 10 with eight, and none of the four with ten within two hours. The bottleneck is feasibility rather than optimality; the remaining gaps on returned plans are 12–36%. Second, where a replayed plan exists both controllers are close to it: the heuristic's makespan averages 1.00, 1.07 and 1.07 times the plan for four, six and eight destinations, and the learned policy's per-instance median averages 0.94, 0.99 and 0.91 times the plan. Both are sometimes faster than the plan, including on instances where the lattice optimum is certified, which is not a contradiction: the lattice restricts where a drone may be, so its optimum is an upper bound on the continuous optimum and a continuous controller can beat it. Third, the learned policy's samples are wide. Its median is at or below the heuristic on most instances, but its range over 30 samples spans up to a factor of five on a single instance (1,142 against a median of 206 on one $M = 8$ instance), which is the same tail behaviour as in Section 6.3. The comparison in RQ3 is therefore against a heuristic that is within a few percent of the best plan an exact method can produce at this scale, and the reference stops being available exactly where the controllers are needed.

6.3. RQ3: Performance across environments

Fleet size divides the configuration space. Table 3 and Figure 3 give the paired comparison across the nine configurations. With three drones the heuristic is faster by 39 and 30 steps in paired median with 20 and 30 destinations. With four drones the two controllers tie: on the standard configuration the median paired difference over 480 rollouts is +2 steps with 234 wins against 239 ($p = 0.52$), and on 4/80 the learned policy is 38 steps faster. With five or more drones the learned policy is faster, by 62 steps at five drones growing to 178 at ten and 76 at twelve (70 wins against 10). The boundary therefore lies at four drones, and the advantage persists beyond the largest fleet we measured with the monotone growth stopping between ten and twelve. An earlier measurement against the heuristic's pre-tuning escape parameters gave the same picture (+40, +0, −54, −170 on 3/30, 4/50, 5/50 and 10/300), so the boundary does not depend on the tuning.

Figure 4 shows the full distributions on the three main configurations: with three drones the heuristic's curve lies to the left, with four the curves coincide, and with five the learned policy's curve lies to the left and reaches completion earlier.

Generalisation from small fleets. The learned policy never saw more than four drones or more than 80 destinations in training, yet it completes every instance with 5 to 12 drones and 50 to 400 destinations and is faster than the heuristic on all of them. Two properties carry this. The set-based encoder has no parameter that depends on N or M, so larger fleets are simply larger sets; and training samples the fleet size and depot afresh every episode. Table 4 separates the two by running the ablation policy that was trained on the fixed 4/50 configuration alone (no configuration randomisation) on the same large-fleet instances without retraining. That policy also completes every instance from 5 to 10 drones, so completion generalises through the architecture alone; but it is slower than the randomised policy on every configuration (a tie against the heuristic at 5/50 against −62, and −51 to −174 against −122 to −178 beyond), so speed on large fleets is learned from the variety of small ones. The same ablation loses 646 steps on 3/30 (Table 8), so randomisation is what makes the small-fleet side usable at all. Both policies were trained with at most four drones; the hop-depth and slot features are normalised by a constant of eight, so fleets of ten and twelve drones also take the encoder outside its training range of feature values, and the policy still completes them.

Tails follow the same split. Where drones are scarce the learned policy's worst case is three to four times the heuristic's (maximum 5,395 versus 1,447 steps on 3/30); where they are plentiful it is lighter (1,101 versus 1,734 on 5/50, 1,365 versus 1,753 on 10/300). On the standard configuration over 480 rollouts against the pre-tuning heuristic the learned policy exceeded 1,500 steps five times and the heuristic fourteen times.

Mechanism. The two controllers differ in one decision. On 3/30 the heuristic stations 1.39 relays on average and the learned policy 0.78; the learned fleet therefore spends 27% of drone-steps stalled by the projection against 8% for the heuristic. Overriding only the learned policy's relay assignment with the heuristic's, at inference, recovers the full 44-step gap; matching only the *number* of relays recovers 14 steps of median but removes the tail (maximum from 4,904 to 1,083). The identity of the relaying drone drives the median and the count drives

Table 2: Lattice MILP reference versus the proposed controllers on instances with three drones (makespan in steps, means over instances). "plan / replayed / optimal" counts instances where the solver returned a plan, where that plan replayed to completion in the simulator, and where the lattice optimum was certified (gap 0); "gap" is the mean remaining optimality gap on returned plans and "min" the mean solver time. The learned column gives the mean of per-instance medians over 30 samples and the overall min–max. The ratio column divides the heuristic and the learned median by the replayed plan, averaged over replayed instances.

M	inst.	plan / replayed / optimal	gap	min	MILP replay	heuristic	learned median (range)	ratio h / l
4	10	9 / 9 / 4	12%	24	112	114	109 (67–250)	1.00 / 0.94
6	10	8 / 8 / 0	34%	60	149	159	150 (103–415)	1.07 / 0.99
8	10	3 / 2 / 0	36%	120	162	188	171 (119–1142)	1.07 / 0.91
10	4	0 / 0 / 0	—	120	—	224	236 (183–427)	—

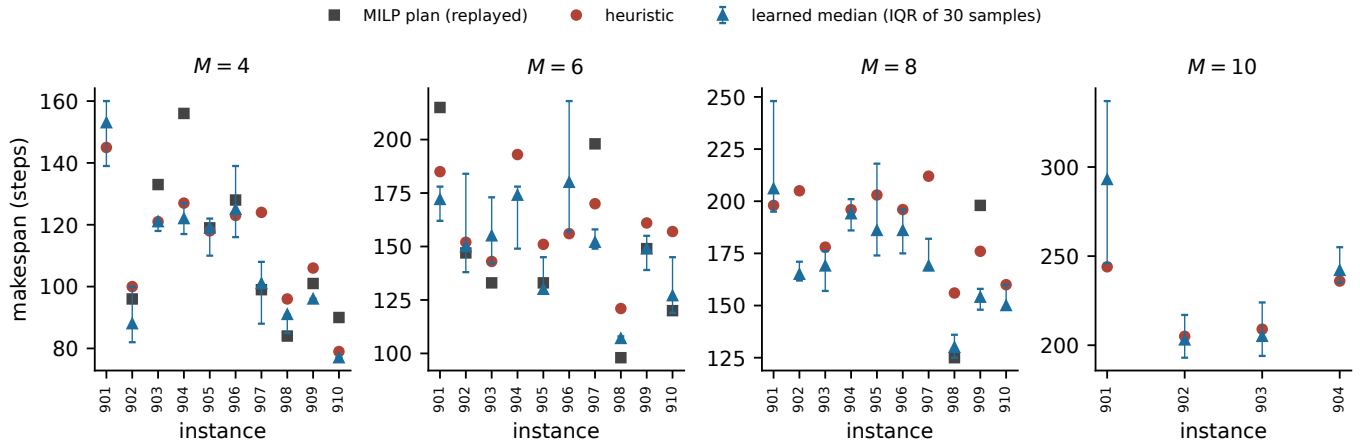


Figure 2: Per-instance makespan on the reference instances with three drones: replayed MILP plan (square, absent where no plan was returned or the replay did not complete), heuristic (circle), and the learned policy's median with the interquartile range of 30 samples (triangle).

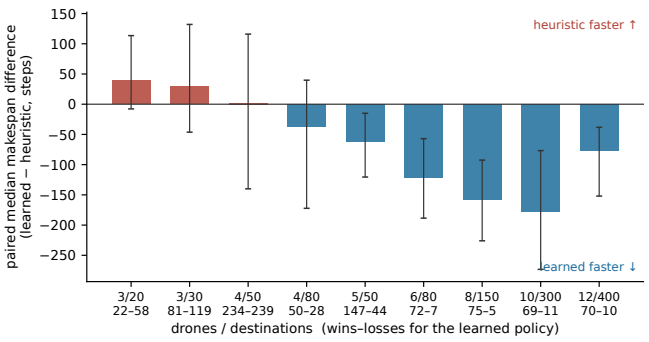


Figure 3: Paired median makespan difference (learned minus heuristic) by configuration, with the interquartile range of the paired differences as error bars and the learned policy's wins–losses under each label. Negative values favour the learned policy; the boundary lies at four drones.

the tail. The information needed to make the heuristic's choice is present in the learned policy's observation: a two-layer classifier predicts the heuristic's relay assignment from the 41 observed features with 85% accuracy, and behaviour cloning of the full rule reaches 95% on relay decisions. When drones are plentiful the opposite happens: the heuristic keeps stationing a chain that the geometry no longer needs, while the learned policy exploits the slack and, with its sampled inference, escapes the stalls that the deterministic rule repeats; this is also where the heuristic loses its few completions.

Why the scarce-fleet deficit resists policy-side fixes. Table 5 lists ten interventions, each trained with four or two seeds under the same recipe and evaluated with the same paired protocol, that attempt to make the learned policy relay like the heuristic when drones are scarce. None improves the 3/30 median and several hurt the configurations where the learned policy wins. The pattern is informative. Adding reward for relaying turns the means into the end: the policy relays for its own sake and 5/50 reverses from −54 to +60. Every route to per-drone credit is blocked for a different reason: a joint-action critic inherits the variance of a sampled teammate action in its target; a monotonic mixing network either diverges when its weights are free or collapses to the mean when they are floored; a sequential single-agent reformulation removes the decomposition but leaves an undiscounted credit chain with no contraction. Initialising the policy by behaviour cloning of the rule, with 95% relay accuracy, is erased within 400 episodes: once the actor is updated, relay usage falls to the level of a randomly initialised policy within 50 episodes and stays there. A penalty on the bottleneck destination's estimated arrival makes the policy reduce the *average* rather than the maximum. We conclude that, within this policy structure and objective, under-relaying is the learner's fixed point, not an artefact of credit, information or initialisation.

Sensitivity to the connectivity enforcement model. The projection of Section 4.1 scales a blocked displacement along its own direction, so a drone pulling on the chain effectively stops. We

Table 3: Paired comparison on identical instances. Positive differences mean the learned policy is slower than the heuristic. p from the Wilcoxon signed-rank test on both-completed pairs.

N/M	rollouts	complete h/l	median heuristic / learned	paired median	win–loss	p
3/20	80	80/80	364 / 420	+39	22–58	$< 10^{-4}$
3/30	200	200/200	572 / 618	+30	81–119	3.4e-04
4/50 (standard)	480	473/480	878 / 878	+2	234–239	0.521
4/80	80	80/80	1,120 / 1,036	−38	50–28	0.006
5/50	200	191/200	588 / 534	−62	147–44	$< 10^{-4}$
6/80	80	79/80	737 / 614	−122	72–7	$< 10^{-4}$
8/150	80	80/80	916 / 722	−158	75–5	$< 10^{-4}$
10/300	80	80/80	1,216 / 1,052	−178	69–11	$< 10^{-4}$
12/400	80	80/80	1,199 / 1,108	−76	70–10	$< 10^{-4}$

Table 4: Generalisation to fleets larger than any seen in training, without retraining. Cells give the learned policy's completions and the paired median difference against the heuristic. The proposed policy is trained on $N \in \{3, 4\}$ with random depots; the comparison policy is the ablation trained on the fixed 4/50 configuration only (two seeds).

N/M	trained on $N \in \{3,4\}$, $M \in [20, 80]$	trained on 4/50 only, seed 1	seed 2
5/50	200/200 / −62	80/80 / +7	80/80 / +44
6/80	80/80 / −122	80/80 / −56	80/80 / −51
8/150	80/80 / −158	80/80 / −129	80/80 / −110
10/300	80/80 / −178	80/80 / −152	80/80 / −174

implemented an alternative that clamps the target position to the nearest point of the communication disc around the binding neighbour, which removes only the outward component and keeps the tangential one (a drone can still circle at constant range). Both controllers use the same projection in each comparison, and connectivity is still never violated. These measurements predate the heuristic tuning of Section 6.4 and use $k_{\text{stall}} = 60$, $h_{\text{hold}} = 40$ for both projections, so they are internally consistent. Evaluating the same learned weights under the sliding projection without retraining reduces the 3/30 deficit from +40 to +21; retraining four seeds under it gives +15 to +48 (mean +28) and lowers the learned fleet's stall ratio from 0.27 to 0.17–0.22, with completion still 100%. The heuristic gains little with three drones (median 570 to 571) but gains more than the learned policy with four (866 to 812 against 879 to 850), so the standard-configuration tie becomes +26 in its favour. We therefore keep the radial projection as the primary model and report this as a sensitivity result: about half of the three-drone deficit is a property of how the constraint is enforced rather than of the learner.

Communication radius. Both controllers were run without retraining at $R = 250$ m and $R = 350$ m (Table 6, Figure 5); the learned policy normalises distances by R, and the instance generator rescales the reachable region $0.9NR$ with R, so both controllers see the same instances at each radius. Completion transfers: the learned policy completes every instance at both radii, while the heuristic loses 18 of 200 instances on 5/50 at the shorter radius. Speed shows two effects. The first is the regime shift seen with fleet size: at $R = 250$ m a destination needs more hops, the fleet is effectively scarcer, and every configuration moves towards the heuristic (+77 on 3/30, +68 on 4/50, and the 5/50 advantage shrinks to a tie at −16, $p = 0.33$); at $R = 350$ m the fleet is effectively more plentiful and the learned policy is faster already with four drones (−40 on 4/50). The second is a training-radius effect: on 3/30 and 5/50 the learned policy's margin is best at the radius it was trained on and worse on both sides (+30 at 300 m against +77 and +52; −62 against −16 and −42), so the normalisation by R transfers completion but not the full speed. The regime variable is therefore the chain demand, the hops a destination needs relative to the drones available, rather than the fleet size alone, and a deployment at a radius far from the training radius should retrain or at least re-evaluate the boundary.

6.4. RQ4: Are the components necessary?

Heuristic. Table 7 grids $k_{\text{stall}} \in \{30, 60, 90, 120\}$, $h_{\text{hold}} \in \{20, 40, 80\}$ and the two escape modes, plus no escape, on 3/30, 4/50 and 5/50 with 40 tuning seeds each, and ranks settings by total completions first and the sum of median makespans second; this criterion was fixed before the measurement. The escape mechanism is necessary: without it the rule completes 26 of 40 standard instances. The mode matters more than the timing: every lead-swap setting completes 117–120 of 120 instances, while retreat settings complete 100–117 because slackening the chain does not change the assignment that caused the stall. Within the lead-swap family the medians differ by at most 40 steps summed over three configurations, which is within the noise of 40 instances; the selected setting ($k_{\text{stall}} = 120$, $h_{\text{hold}} = 20$) is the only one that completes all 120 and also has the lowest median sum. The adaptive relay count is necessary as well: fixing one or two relays completes 0–65% (Table 1).

Learned policy. Table 8 removes one ingredient at a time, retrains two seeds and evaluates on the three main configurations; values are makespan differences from the proposed recipe. Four

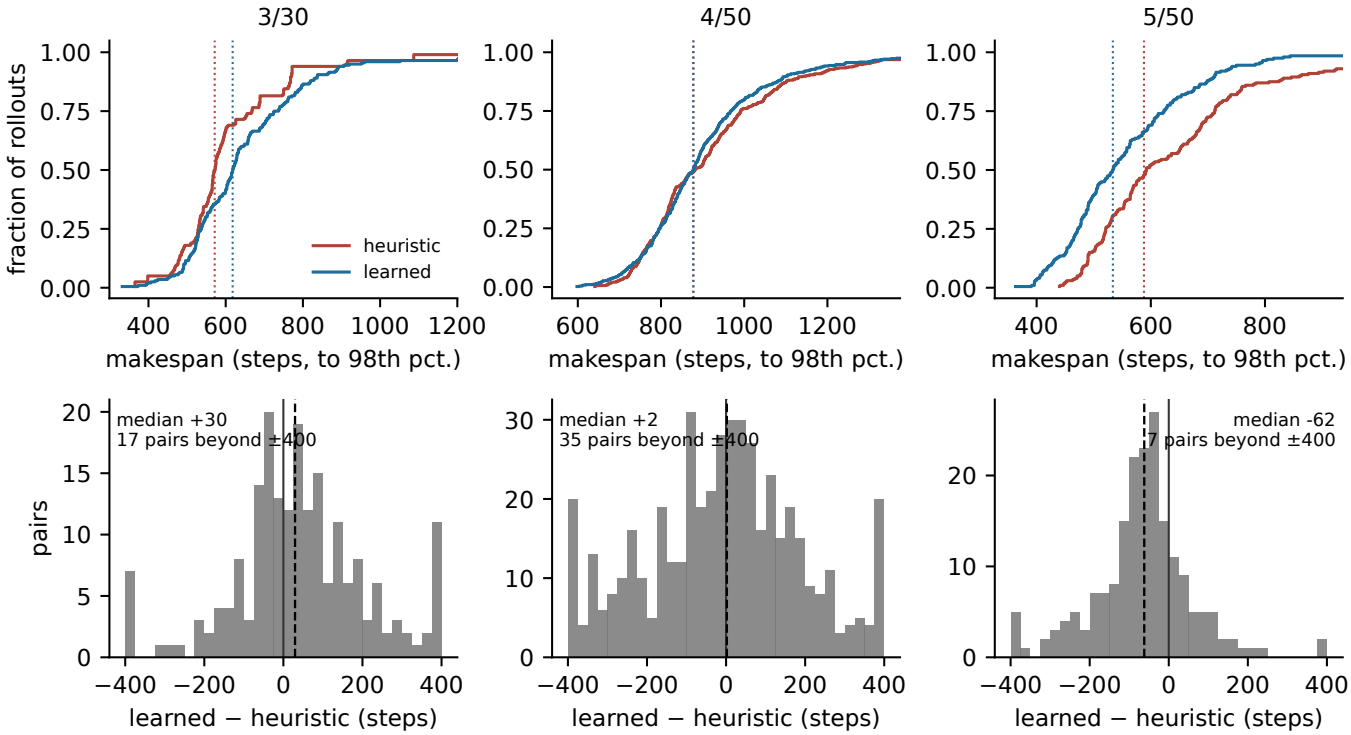


Figure 4: Top: cumulative distribution of makespan on 3/30, 4/50 and 5/50 (dotted lines mark medians; axes cut at the 98th percentile). Bottom: distribution of the paired difference on both-completed instances, clipped to ±400 steps with the number of clipped pairs noted.

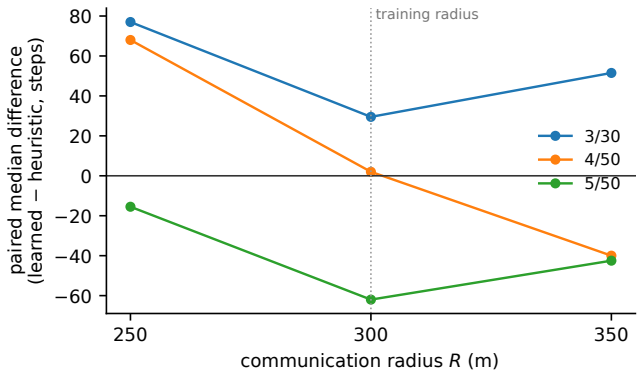


Figure 5: Paired median difference against the communication radius without retraining (training radius 300 m, dotted). A shorter radius moves every configuration towards the heuristic; a longer radius helps the learned policy on 4/50 but not beyond its training-radius margin on 3/30 and 5/50.

independent trainings of the same recipe span 919 to 979 steps on the standard configuration, so a two-seed mean is meaningful only beyond 70–80 steps. Rule regularisation and configuration randomisation are necessary; entropy annealing is the single largest contributor on the standard configuration; autoregressive decisions help with four drones and more; the residual penalty is neutral. A manager with a fixed-size MLP encoder instead of the set encoder, trained in an earlier version on the standard configuration only, completed 38% of standard instances and could not be run on other fleet sizes at all.

6.5. *Inference cost*

Table 9 gives the latency of one manager decision on a CPU. The heuristic costs 0.05–0.12 ms; the learned policy 1.3–2.5 ms without autoregression and 2.7–7.4 ms with it, growing linearly with the fleet because each drone is a sequential network call. Over a 900-step episode the learned policy's cumulative decision time is 0.16 s. The lattice MILP takes 600 to 5,400 seconds on instances with four to ten destinations and does not return a plan for ten.

7. Discussion

Deployment. The two controllers are complementary and the boundary is sharp, so a fleet operator can choose by fleet size: the heuristic with three drones, the learned policy with five or more, where it is both faster and more reliable; with four the two tie and the learned policy completes more. Both decide in milliseconds and both keep the fleet connected by construction.

What the negative results mean. Ten interventions that change how the learner is credited, informed, initialised or rewarded leave the scarce-fleet deficit in place, and a change to the environment's enforcement model removes half of it. The deficit is therefore best understood as an interaction between the learner's fixed point under this objective and the geometry of the constraint, not as a shortcoming that more training or a

Table 5: Interventions that did not remove the scarce-fleet deficit. Paired median against the heuristic (positive is slower); the proposed recipe is the reference row. These runs were evaluated against the heuristic with its pre-tuning escape parameters ($k_{\text{stall}} = 60$, $h_{\text{hold}} = 40$), so the column values are comparable to the reference row of this table but not numerically identical to Table 3, which uses the tuned heuristic.

Intervention	3/30	5/50	4/50	Mechanism of failure
Proposed recipe (reference)	+40	−54	+0	—
Relay-weighted rule regularisation	+54, +56	−44, −19	+41, +54	relay-specific regularisation harms
Per-drone relay credit	+36 to +82	+60 to +106	+39 to +131	relaying becomes the goal
Checkpoint selection on 3/30 (8 seeds)	+58	−42	+10	improves only the standard config.
Zero-sum credit redistribution	+91, +126	+98, +172	+146, +180	same, larger
Joint-action critic	+89	−27	+58	target variance from sampled teammates
Near-first decision order	+46, +100	−43, −38	+4, +106	order is not the cause
Re-decision on stall (learned)	+82, +95	−38, −34	+17, +55	irregular option lengths
Monotonic mixing critic	+145	+5	+139	divergence vs. credit on one axis
Sequential single-agent credit	+142	+17	+116	no contraction within a decision
Behaviour-cloning initialisation	+42 to +69	−42 to −20	−4 to +40	erased within 400 episodes
Bottleneck-arrival penalty	+105	−8	+81	reduces the mean, not the maximum

Table 6: Transfer to other communication radii without retraining. Cells give completions (heuristic/learned) and the paired median difference with win–loss and p; $R = 300$ m is the training radius.

N/M	$R = 250$ m	$R = 300$ m	$R = 350$ m
3/30	200/200 / +77 (45–154, $< 10^{-4}$)	200/200 / +30 (81–119, 3.4e-04)	200/200 / +52 (58–141, $< 10^{-4}$)
4/50	118/120 / +68 (40–77, $< 10^{-4}$)	473/480 / +2 (234–239, 0.521)	119/120 / −40 (77–42, 1.1e-04)
5/50	182/200 / −16 (99–83, 0.327)	191/200 / −62 (147–44, $< 10^{-4}$)	200/200 / −42 (152–46, $< 10^{-4}$)

better critic would fix. For a practitioner the actionable consequence is the one above: pick the controller by regime.

Limitations. The learned policy was trained at a single communication radius; it transfers to radii of 250 and 350 m in completion but its speed ranking against the heuristic shifts with the radius (Section 6.3), and radii outside this range were not tested. The environment is planar, ignores wind, battery and collisions, and uses a disc communication model; real links are not discs. The MILP reference is a lattice restriction with a remaining optimality gap, not an exact optimum. The heuristic's novelty lies in the escape mechanism and the adaptive relay count rather than in the chain geometry itself.

Methodological notes. Three practices mattered for the conclusions. Completed-episode averages favour the controller that gives up, so comparisons must be paired. Forty rollouts cannot support claims about tails; several of our own interim conclusions reversed when the sample grew. And a pre-check that uses a different metric from the final judgement will pass interventions that the final judgement rejects.

8. Conclusion

We proposed two controllers for connectivity-constrained drone delivery, a geometric chain-relay heuristic with deadlock escape and a fleet-size-agnostic learned policy, both acting through a projection that keeps the fleet connected at every step. Under a paired protocol across fleets of three to twelve drones the learned policy completes every instance and is faster by 38 to 178 steps once more than four drones are available, although it was trained on three and four, while the heuristic is faster by 30–39 steps with three. The scarce-fleet deficit is one decision, which drone relays; it resists ten policy-side interventions and halves when the enforcement model is changed. Both controllers are three orders of magnitude faster than exact optimisation on a lattice, which stops returning plans at ten destinations. The natural next step is a policy whose output is a team assignment rather than a set of per-drone options, so that the relay identity becomes a coordinate the critic can value directly.

Acknowledgements

[To be added.]

Data and code availability

Code, trained weights, instance generators and the paired-comparison records are available at `https://github.com/koptimizer/relay_drone_network`.

References

[1] C. C. Murray, A. G. Chu, The flying sidekick traveling salesman problem: Optimization of drone-assisted parcel delivery, Transportation Research Part C: Emerging Technologies 54 (2015) 86–109. doi:10.1016/j.trc.2015.03.005.

[2] C. C. Murray, R. Raj, The multiple flying sidekicks traveling salesman problem: Parcel delivery with multiple drones, Transportation Research Part C: Emerging Technologies 110 (2020) 368–398. doi:10.1016/j.trc.2019.11.003.

[3] N. Agatz, P. Bouman, M. Schmidt, Optimization approaches for the traveling salesman problem with drone, Transportation Science 52 (4) (2018) 965–981. doi:10.1287/trsc.2017.0791.

Table 7: Heuristic ablation on tuning seeds disjoint from the evaluation seeds (40 instances per configuration). Cells give completed instances and the median makespan over completed instances; the last column sums completions over the three configurations.

mode	k_{stall} / h_{hold}	3/30	4/50	5/50	completions
none	—	39/40 / 561	26/40 / 805	36/40 / 613	101/120
lead swap	30 / 20	40/40 / 566	38/40 / 827	40/40 / 630	118/120
lead swap	30 / 40	40/40 / 560	39/40 / 847	40/40 / 625	119/120
lead swap	30 / 80	40/40 / 565	39/40 / 892	38/40 / 628	117/120
lead swap	60 / 20	40/40 / 562	40/40 / 879	39/40 / 631	119/120
lead swap	60 / 40	40/40 / 561	39/40 / 852	40/40 / 624	119/120
lead swap	60 / 80	40/40 / 561	38/40 / 838	40/40 / 636	118/120
lead swap	90 / 20	40/40 / 562	39/40 / 865	40/40 / 624	119/120
lead swap	90 / 40	40/40 / 565	39/40 / 887	38/40 / 594	117/120
lead swap	90 / 80	40/40 / 560	39/40 / 891	40/40 / 637	119/120
lead swap	**120 / 20**	40/40 / 568	40/40 / 826	40/40 / 603	**120/120**
lead swap	120 / 40	40/40 / 565	39/40 / 869	39/40 / 614	118/120
lead swap	120 / 80	40/40 / 558	38/40 / 834	40/40 / 621	118/120
retreat	30 / 20	37/40 / 560	38/40 / 810	37/40 / 605	112/120
retreat	30 / 40	38/40 / 558	33/40 / 824	38/40 / 637	109/120
retreat	30 / 80	37/40 / 557	27/40 / 812	38/40 / 608	102/120
retreat	60 / 20	39/40 / 566	36/40 / 796	40/40 / 622	115/120
retreat	60 / 40	39/40 / 550	39/40 / 833	39/40 / 613	117/120
retreat	60 / 80	39/40 / 560	25/40 / 823	36/40 / 634	100/120
retreat	90 / 20	39/40 / 557	38/40 / 829	39/40 / 606	116/120
retreat	90 / 40	39/40 / 563	39/40 / 855	39/40 / 616	117/120
retreat	90 / 80	39/40 / 566	35/40 / 838	40/40 / 641	114/120
retreat	120 / 20	39/40 / 556	39/40 / 869	38/40 / 626	116/120
retreat	120 / 40	39/40 / 558	36/40 / 873	38/40 / 608	113/120
retreat	120 / 80	39/40 / 556	37/40 / 863	40/40 / 616	116/120

Table 8: Ablation of the learned policy: makespan difference from the proposed recipe over completed episodes, two seeds. Positive means the removed component was needed. Differences below 70–80 steps are within seed variance.

Removed component	4/50	3/30	5/50
Rule regularisation	+320	+408	+92
Configuration randomisation	−44	+646	+68
Entropy target annealing	+128	+19	+61
Autoregressive decisions	+94	−46	+33
Residual penalty	−28	−37	+8

[4] K. Dorling, J. Heinrichs, G. G. Messier, S. Magierowski, Vehicle routing problems for drone delivery, IEEE Transactions on Systems, Man, and Cybernetics: Systems 47 (1) (2017) 70–85.

[5] J. Park, C. Kwon, J. Park, Learn to solve the min-max multiple traveling salesmen problem with reinforcement learning, in: Proceedings of the 22nd International Conference on Autonomous Agents and Multiagent Systems, 2023, pp. 878–886.

[6] Y. Guo, Z. Ren, C. Wang, iMTSP: Solving min-max multiple traveling salesman problem with imperative learning, arXiv:2405.00285 (2024).

[7] Y. Zhu, H. Xu, A. A. Malikopoulos, N. Geroliminis, Dual-mission drone fleets: Delivery with secondary sensing tasks, arXiv:2604.02471 (2026).

[8] Y. Zeng, R. Zhang, T. J. Lim, Throughput maximization for UAV-enabled mobile relaying systems, IEEE Transactions on Communications 64 (12) (2016) 4983–4996. doi:10.1109/TCOMM.2016.2611512.

[9] E. Yanmaz, Positioning aerial relays to maintain connectivity during drone team missions, Ad Hoc Networks 128 (2022) 102800.

[10] R. Ding, J. Chen, W. Wu, J. Liu, F. Gao, X. Shen, Packet routing in dynamic multi-hop UAV relay network: A multi-agent learning approach, IEEE Transactions on Vehicular Technology 71 (9) (2022) 10059–10072.

[11] MUTTO: Multi-agent reinforcement learning for joint trajectory and transmit power optimisation of relay and mission UAVs, Vehicular Communications (2024).
URL https://www.sciencedirect.com/science/article/abs/pii/S1570870524002130

[12] MAEN: Multi-agent aerial relay networking, Proceedings of ACM MobiCom (2024). doi:10.1145/3636534.3694730.

[13] M. Li, Y. Jie, Y. Kong, H. Cheng, Decentralized global connectivity maintenance for multi-robot navigation: A reinforcement learning approach, arXiv:2109.08536 (2021).

[14] J. De Hoog, S. Cameron, A. Visser, Role-based autonomous multi-robot exploration, in: Proceedings of the International Conference on Advanced Cognitive Technologies and Applications, 2009.

[15] K. Cesare, R. Skeele, S.-H. Yoo, Y. Zhang, G. Hollinger, Multi-UAV exploration with limited communication and battery, in: Proceedings of the IEEE International Conference on Robotics and Automation, 2015, pp. 2230–2235. doi:10.1109/ICRA.2015.7139494.

[16] T. Haarnoja, A. Zhou, P. Abbeel, S. Levine, Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor, in: Proceedings of the 35th International Conference on Machine Learning, 2018, pp. 1861–1870.

[17] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, I. Mordatch, Multi-agent actor-critic for mixed cooperative-competitive environments, in: Advances in Neural Information Processing Systems 30, 2017, pp. 6379–6390.

[18] J. Foerster, G. Farquhar, T. Afouras, N. Nardelli, S. Whiteson, Counterfactual multi-agent policy gradients, in: Proceedings of the 32nd AAAI Conference on Artificial Intelligence, 2018, pp. 2974–2982.

[19] P. Sunehag, G. Lever, A. Gruslys, W. M. Czarnecki, V. Zambaldi, M. Jaderberg, M. Lanctot, N. Sonnerat, J. Z. Leibo, K. Tuyls, T. Graepel, Value-decomposition networks for cooperative multi-agent learning based on team reward, in: Proceedings of the 17th International Conference on Autonomous Agents and Multiagent Systems, 2018, pp. 2085–2087.

[20] T. Rashid, M. Samvelyan, C. Schroeder de Witt, G. Farquhar, J. Foerster, S. Whiteson, QMIX: Monotonic value function factorisation for deep multi-agent reinforcement learning, in: Proceedings of the 35th Interna-

Table 9: Latency of one manager decision (milliseconds, CPU) and the cumulative decision time of a 900-step episode.

Controller	$N = 3$	$N = 4$	$N = 6$	$N = 8$	900 steps
Geometric rule + escape	0.05	0.08	0.10	0.12	0.003 s
Learned, independent decisions	1.27	1.52	1.94	2.45	0.069 s
Learned, autoregressive (proposed)	2.70	3.52	5.32	7.35	0.158 s

tional Conference on Machine Learning, 2018, pp. 4295–4304.

[21] R. S. Sutton, D. Precup, S. Singh, Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning, Artificial Intelligence 112 (1–2) (1999) 181–211.

[22] S. Huang, S. Ontañón, A closer look at invalid action masking in policy gradient algorithms, in: Proceedings of the International FLAIRS Conference, Vol. 35, 2022.

[23] G. Dalal, K. Dvijotham, M. Vecerik, T. Hester, C. Paduraru, Y. Tassa, Safe exploration in continuous action spaces, arXiv:1801.08757 (2018).

[24] A. Y. Ng, D. Harada, S. Russell, Policy invariance under reward transformations: Theory and application to reward shaping, in: Proceedings of the 16th International Conference on Machine Learning, 1999, pp. 278–287.

[25] Gurobi Optimization, LLC, Gurobi optimizer reference manual, version 13, `https://www.gurobi.com` (2026).